Unconstrained Nonconvex Optimization for Just-in-Time Defect Prediction

XUE, Pengyu
Student ID: 26022951R
The Hong Kong Polytechnic University

Abstract

Just-in-time defect prediction ranks commits for review and testing, yet the optimizer and stopping rule behind its ranker are chosen by convention, not by optimization evidence. On a class-weighted one-hidden-layer model with an unconstrained nonconvex objective, the zero parameter vector is a strict saddle for all five datasets, so a vanishing gradient does not certify a minimum, and an Armijo analysis gives a step-size lower bound and first-order stationarity. Comparing first-order descent, momentum, adaptive moments and limited-memory quasi-Newton over 240 tuning and 96 reporting runs, L-BFGS attains the lowest final training objective on every task, while test average precision favors gradient descent on four tasks and Adam on Flink. Because average precision is invariant to strictly increasing score transformations, this divergence reflects reordering of commits: in 85 of 96 runs, iterating past the validation checkpoint lowers training loss and test AP alike. Objective reduction, stationarity and ranking quality therefore measure different things, and the optimizer and stopping rule should be selected against ranking quality rather than the training loss.

1 Introduction

Just-in-time defect prediction ranks incoming commits for review and testing. Its predictor is trained by an optimizer, and two decisions are usually settled by convention: which optimizer to use, and when to stop. Both decisions are commonly justified by the training objective, on the assumption that a lower objective and a smaller gradient indicate a better ranker. This assumption is rarely tested, and it can fail, because the training objective and commit ranking respond to different properties of the predictor.

We examine this assumption on a class-weighted one-hidden-layer model trained under an unconstrained nonconvex objective. The model is small enough for exact analysis but retains the structure that matters: a logistic loss composed with a neural parameterization, so that the objective is nonconvex even though the loss is convex in the logit. We first characterize the landscape. The zero parameter vector is a strict saddle for every dataset we study, certified by a negative-curvature direction derived from class-mean contrast, which means that a vanishing gradient does not certify a minimum. An Armijo backtracking analysis then gives a step-size lower bound and a first-order stationarity guarantee.

We then compare four update rules, first-order descent, heavy-ball momentum, adaptive moments and limited-memory quasi-Newton, under a matched tuning protocol across three Apache projects and two external datasets. The comparison separates three quantities that are often conflated: reduction of the training objective, approach to first-order stationarity, and test ranking quality measured by average precision. L-BFGS attains the lowest final training objective on every task, while test average precision favors gradient descent on four tasks and Adam on Flink. The divergence is not noise. Because average precision is invariant to strictly increasing score transformations, a decline

COMP6704 Individual Assignment, Fall 2026.

in average precision can only come from reordering of positive and negative commits, and this reordering occurs in 85 of 96 runs when training continues past the validation-selected checkpoint.

These results support a simple conclusion. Objective reduction, stationarity and ranking quality measure different things, and in this setting they can move in opposite directions. Optimizer choice and stopping rule should therefore be decided against the ranking criterion rather than the training objective.

2 Related work

Software defect prediction and data quality. Work on just-in-time defect prediction has largely standardized the data side of the problem. ApacheJIT provides a large corpus of changes with process and change metrics [Keshavarz and Nagappan, 2022], longitudinal studies of Qt and OpenStack show that predictive relationships can shift as projects evolve [McIntosh and Kamei, 2018], and analyses of SZZ variants show that label construction affects both prediction and interpretation [Fan et al., 2021]. Consequently, current practice is careful about labels and chronological splits, but comparatively silent about training itself: the optimizer and the stopping rule are typically fixed by convention or by the training objective, on the assumption that a lower objective yields a better ranker. This assumption has not been tested for this task.

Optimization methods and theory. The optimization literature offers both the algorithms we compare and the criteria by which they are usually judged. Sufficient-decrease search [Armijo, 1966], heavy-ball momentum [Polyak, 1964], adaptive moments [Kingma and Ba, 2015] and limited-memory curvature approximation [Liu and Nocedal, 1989] are standard representatives of four update rules, and their convergence has been analyzed under non-uniform smoothness [Wang et al., 2024], global smoothness and second-moment conditions [Jin et al., 2025], and difficult landscapes in scientific machine learning [Rathore et al., 2024]. These analyses share a common criterion: success is measured by gradient norms and objective values. Rankings based on such criteria are themselves sensitive to search spaces, tuning protocols and tasks [Choi et al., 2019, Schmidt et al., 2021]. What remains open is whether optimizers that win on these criteria also produce better defect rankings, and our saddle certificate shows that a vanishing gradient can occur at a non-minimizing point, so the two questions cannot be assumed equivalent.

3 Objective and structural analysis

3.1 A smooth unconstrained learning problem

For fixed preprocessed training data $\{(x_i, y_i)\}_{i=1}^n$, let $x_i \in \mathbb{R}^d$ and $y_i \in \{0, 1\}$. A one-hidden-layer network uses parameters $\theta = (W, b, v, c)$, with $W \in \mathbb{R}^{d\times h}$, $b, v \in \mathbb{R}^h$, and $c \in \mathbb{R}$:

$$h_i = \tanh(W^\top x_i + b), \qquad z_i = v^\top h_i + c, \qquad p_i = \frac{1}{1 + \exp(-z_i)}. \tag{1}$$

Here $d = 12$, $h = 32$, and the parameter count is $p = dh + 2h + 1 = 449$. Let $n_0, n_1 > 0$ denote training-class counts. Define $a_i = 1$ for negative examples and $a_i = n_0/n_1$ for positive examples, $A = \sum_i a_i$, and $q_i = a_i/A$. The common objective is

$$F(\theta) = \sum_{i=1}^{n} q_i \left[\log(1 + \exp(z_i)) - y_i z_i\right] + \frac{\lambda}{2}\|\theta\|_2^2, \qquad \lambda = 10^{-4}. \tag{2}$$

All parameters, including biases, are regularized. The weights are fixed from training data and sum to one. They give each class total weight $1/2$, independent of its frequency. This changes the fitted probability scale; classification thresholds are selected on validation data. The parameter space is unconstrained, and the objective is smooth and nonconvex.

3.2 Gradient and curvature

Write $\delta_i = q_i(p_i - y_i)$ and $u_i = \delta_i v \odot (1 - h_i \odot h_i)$, where $\odot$ is coordinatewise multiplication. The analytic gradient used by every method is

$$\begin{aligned} \nabla_W F &= \sum_i x_i u_i^\top + \lambda W, \quad & \nabla_b F &= \sum_i u_i + \lambda b, \\ \nabla_v F &= \sum_i \delta_i h_i + \lambda v, & \nabla_c F &= \sum_i \delta_i + \lambda c. \end{aligned} \tag{3}$$

With $J_i = \nabla_\theta z_i$, the Hessian is

$$\nabla^2 F = \sum_i q_i \left[p_i(1-p_i) J_i J_i^\top + (p_i - y_i) \nabla^2 z_i \right] + \lambda I. \tag{4}$$

Although logistic loss is convex in a scalar logit, the second term in brackets can be indefinite. A local minimum requires both a zero gradient and a positive-semidefinite Hessian; the following calculation exhibits a stationary point violating the latter condition for our actual datasets.

3.3 A data-specific strict saddle

At $\theta = 0$, all hidden activations and logits vanish and $p_i = 1/2$. Every parameter gradient except that of c vanishes by the network's product structure; the remaining gradient is $\sum_i q_i(1/2 - y_i) = 0$ by class balancing. Hence the origin is a stationary point with $F(0) = \log 2$. Define the preprocessed class means μ_0, μ_1 and

$$r = \sum_i q_i(1/2 - y_i) x_i = \frac{\mu_0 - \mu_1}{4}. \tag{5}$$

Restrict perturbations to the input-weight column $W_{:j}$ and output weight v_j of any one hidden unit. The corresponding principal Hessian block at zero is

$$B_j = \begin{pmatrix} \lambda I_d & r \\ r^\top & \lambda \end{pmatrix}, \qquad \lambda_{\min}(B_j) = \lambda - \|r\|_2. \tag{6}$$

Indeed, for $r \neq 0$, the unit direction $(r/\|r\|_2, -1)/\sqrt{2}$ has curvature $\lambda - \|r\|_2$. If $\|r\|_2 > \lambda$, extending this direction by zeros supplies a negative-curvature direction of the complete objective. Perturbing a hidden bias alone gives positive curvature λ. Consequently the stationary origin is a strict saddle. The certificate uses a principal subspace of the 449-dimensional Hessian.

Let d_- be the extended unit direction above. Taylor expansion gives $F(td_-) = F(0) + \frac{1}{2}t^2(\lambda - \|r\|_2) + o(t^2) < F(0)$ for sufficiently small nonzero t. The certificate holds at any positive hidden width with unchanged preprocessing and weighting.

Table 1 evaluates the certificate. All five values are negative, with zero-point gradient residuals below 4.8×10^{-16}. Zero initialization would therefore satisfy a first-order stopping rule immediately at a saddle. Paired random initialization starts away from this exact saddle. The certificate illustrates the distinction between first-order stationarity and minimum curvature.

The negative direction has an architectural interpretation. At the origin, changing an input-weight column alone leaves its output weight at zero, and changing the output weight alone leaves its hidden activation at zero. Either isolated perturbation pays the positive quadratic regularization cost. Moving the two weights together changes the logit at second order; the resulting mixed curvature is governed by the difference between preprocessed class means. Class balancing makes the output-bias gradient vanish, while that feature contrast produces negative curvature. Thus a convex logistic loss in logit space becomes a nonconvex learning objective through the neural parameterization.

3.4 Existence, smoothness, and a descent guarantee

The logistic terms in (2) are nonnegative. Therefore $F(\theta) \geq (\lambda/2)\|\theta\|_2^2 \to \infty$ as $\|\theta\|_2 \to \infty$. Continuity and this coercivity imply existence of a finite global minimizer. Because softplus and $\tanh$ are infinitely differentiable, F is smooth. Its Hessian is bounded on every compact ball, giving a Lipschitz gradient there. The analysis uses a Lipschitz constant on the bounded sublevel set.

Table 1: Nonconvexity certificate for the actual 32-unit objectives. The all-zero stationary point has a negative-curvature direction; central differences at 10^{-3} and 10^{-4} agree with the analytic curvature. FD error is the larger absolute error.

Task	$\|r\|_2$	$\lambda - \|r\|_2$	$\|g(0)\|_\infty$	FD error
Camel	0.4036	-0.4035	5.2×10^{-18}	1.2×10^{-6}
Ignite	0.5782	-0.5781	3.5×10^{-17}	1.8×10^{-6}
Flink	0.5664	-0.5663	2.8×10^{-17}	1.8×10^{-6}
OpenStack	0.4009	-0.4008	2.1×10^{-17}	8.9×10^{-7}
Qt	0.3620	-0.3619	4.2×10^{-17}	1.2×10^{-6}

For gradient descent, let $g_k = \nabla F(\theta_k)$ and use Armijo backtracking [Armijo, 1966] with fixed initial trial step $\bar{\alpha} > 0$, shrinkage $\rho \in (0, 1)$, and $c_1 \in (0, 1)$. Accepted steps satisfy

$$F(\theta_k - \alpha_k g_k) \le F(\theta_k) - c_1 \alpha_k \|g_k\|_2^2. \tag{7}$$

Accepted iterates remain in the compact initial sublevel set. Their gradients are bounded, and trial step sizes are at most $\bar{\alpha}$; therefore all trial segments lie in a compact ball with a finite gradient-Lipschitz constant L. The descent lemma guarantees acceptance for $\alpha \le 2(1 - c_1)/L$. Applying Armijo at every iteration yields

$$\alpha_k \ge \underline{\alpha} = \min\left\{\bar{\alpha}, \frac{2\rho(1 - c_1)}{L}\right\} > 0, \qquad \min_{0 \le k < K} \|g_k\|_2^2 \le \frac{F(\theta_0) - \inf F}{c_1 \underline{\alpha} K}. \tag{8}$$

Summing (7) also gives $\sum_k \|g_k\|_2^2 < \infty$, hence $\|g_k\|_2 \to 0$. Equation (8) bounds the best squared gradient norm and establishes first-order stationarity for Armijo gradient descent.

4 Optimization algorithms and computational accounting

All solvers use Eq. (2), full-batch float64 gradients, and identical saved Xavier initializations within each dataset, width and seed. Biases start at zero and weights are random, avoiding the exact saddle in Section 3.

Gradient descent with backtracking. For $g_k = \nabla F(\theta_k)$, use $\theta_{k+1} = \theta_k - \alpha_k g_k$. The tuned initial trial step is halved until Eq. (7) holds with $c_1 = 10^{-4}$.

Heavy-ball momentum. With $u_0 = 0$, use $u_{k+1} = \beta u_k + g_k$ and $\theta_{k+1} = \theta_k - \alpha u_{k+1}$ [Polyak, 1964]. The velocity uses the unnormalized heavy-ball convention.

Adam. Adam [Kingma and Ba, 2015] uses moments $m_0 = \nu_0 = 0$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$:

$$\begin{aligned} m_{k+1} &= \beta_1 m_k + (1 - \beta_1) g_k, & \nu_{k+1} &= \beta_2 \nu_k + (1 - \beta_2)(g_k \odot g_k), \\ \widehat{m}_{k+1} &= m_{k+1}/(1 - \beta_1^{k+1}), & \widehat{\nu}_{k+1} &= \nu_{k+1}/(1 - \beta_2^{k+1}). \end{aligned} \tag{9}$$

The update is $\theta_{k+1} = \theta_k - \alpha \widehat{m}_{k+1}/(\sqrt{\widehat{\nu}_{k+1}} + \epsilon)$, coordinatewise. The gradient includes L2 regularization, and the second moment scales coordinates.

Limited-memory BFGS. The two-loop recursion computes $d_k = -H_k g_k$ from at most m pairs $s_k = \theta_{k+1} - \theta_k$, $y_k = g_{k+1} - g_k$ [Liu and Nocedal, 1989]. The underlying inverse-BFGS transformation, with $\rho = (s^\top y)^{-1}$, is

$$H^+ = (I - \rho s y^\top) H (I - \rho y s^\top) + \rho s s^\top. \tag{10}$$

The limited-memory recursion applies retained transformations to a scaled identity, using the latest valid pair's $(s^\top y)/(y^\top y)$ or scale one without history. SciPy's strong-Wolfe search combines sufficient decrease with $|g_{k+1}^\top d_k| \le c_2 |g_k^\top d_k|$, using $(c_1, c_2) = (10^{-4}, 0.9)$ and recorded Armijo fallback on failure. Non-descent directions reset history. Pairs require finite $s_k^\top y_k > 10^{-10} \|s_k\|_2 \|y_k\|_2$ and $s_k^\top y_k > 0$. The recursion and curvature safeguards are implemented directly.

Table 2: Chronological datasets: sample count (positive percentage). Each task has independently fitted training-only preprocessing. OpenStack and Qt are external tasks.

Task	Training	Validation	Test
Camel	13,016 (18.3%)	3,255 (10.7%)	6,429 (5.3%)
Ignite	7,180 (14.9%)	1,796 (19.4%)	3,060 (33.4%)
Flink	5,026 (18.1%)	1,257 (49.6%)	5,408 (23.6%)
OpenStack	7,424 (9.5%)	2,475 (21.1%)	2,475 (15.6%)
Qt	15,090 (9.0%)	5,030 (6.2%)	5,030 (6.6%)

Cost and stopping. An objective/gradient evaluation costs $O(ndh)$ with $O(nh)$ activations. Optimizer state and update overhead are $O(p)$ for GD/momentum/Adam and $O(mp)$ for L-BFGS. Counts N_f, N_g include actual objective and gradient computations, including joint calls but excluding cache hits; $N_f - N_g$ counts loss-only calls. We report both counts and solver time to capture computational work. Stopping requires

$$R_k = \frac{\|g_k\|_\infty}{\max(1, \|g_0\|_\infty)} \leq 10^{-4}, \tag{11}$$

or a recorded resource/failure termination. First-hit summaries at 10^{-2} and 10^{-3} describe intermediate progress; the terminal tolerance is 10^{-4}.

5 Experimental Design

Research questions. RQ1 compares objective reduction, stationarity and cost. RQ2 examines initialization sensitivity and, for Adam/L-BFGS on Flink, network-width sensitivity. RQ3 asks whether better numerical optimization accompanies better defect ranking on later commits.

Data and chronology. ApacheJIT provides 106,674 changes [Keshavarz and Nagappan, 2022], and its pilot results informed the project-level extension. We select the three largest unambiguous projects by sample count, excluding overlapping Hadoop identifiers: Camel, Ignite and Flink. Models are fitted separately. Each project's 2017–2019 commits form the test set; the latest approximately 20% of pre-2017 commits form validation, with the remainder for training. Splits use UTC author timestamps, keep equal boundary timestamps together, and have disjoint identifiers.

Qt and OpenStack use the original authors' replication data [McIntosh and Kamei, 2018], with their released date/change-size filters and chronological 60/20/20 splits, yielding 25,150 and 12,374 changes. Labels are `bugcount`>0, with missing counts treated as nonpositive following the released loader. The model and candidate grids were set using Apache data, and each external task fits a new model with Apache-selected optimizer settings.

The twelve inputs describe change size, modification scope, entropy, developer experience, age and prior changes. Except entropy, inputs receive $\log(1 + x)$; missing values and standardization statistics are computed from training data only. Natural class frequencies are retained, with class weights fixed per training set. Identifiers, dates, fix indicators and labels are excluded.

Selection and reproducibility. Principal models use $h = 32$ and $\lambda = 10^{-4}$. For each Apache project and optimizer, eight prespecified settings are screened with seeds 2026/2027, at most 150 objective calls and 15 seconds; the two eligible candidates with the highest mean validation AP are rerun with both seeds at 1,500 calls and 120 seconds, and mean validation AP selects the project-specific setting. For external tasks, the global setting is the one with the lowest mean rank across the three Apache screening results; this choice is fixed before external training, and external validation selects checkpoints and F1 thresholds. Screening rules, seeds and timing boundaries are fixed before training, and evaluation is outside the solver timer.

The main matrix contains three Apache projects $\times$ four methods $\times$ five paired seeds (10–14), giving 60 runs. External replication adds two tasks $\times$ four methods $\times$ three seeds (10–12), giving 24 runs. Widths 8 and 128 add 12 Flink runs for Adam and L-BFGS, compared against matching width-32 seeds while retaining width-32 settings. All 96 runs stop at the first of 1,500 objective calls, 120

Table 3: Selected optimizer settings from two-seed Apache tuning. External settings minimize mean Apache screening rank.

Scope	GD $\bar{\alpha}$	Momentum (α, β)	Adam α	L-BFGS m
Camel	1	$(1, 0.9)$	0.03	75
Ignite	1	$(0.1, 0.95)$	0.003	3
Flink	100	$(1, 0.9)$	0.0003	20
External (frozen)	30	$(1, 0.9)$	0.01	5

Table 4: Apache optimization outcomes: loss/time are mean $\pm$ sample SD; call counts are rounded means. $R = \|g\|_\infty / \max(1, \|g_0\|_\infty)$; the hit column counts final iterates attaining the tolerance. N_f includes loss-only and loss–gradient calls, N_g counts gradients, and time reports runtime.

Task	Method	F_{end}	$R \le 10^{-4}$	N_f / N_g	Time (s)
Camel	GD	0.5503 ± 0.0012	0/5	1500 / 706	28.4 ± 0.7
	Momentum	0.5156 ± 0.0060	0/5	1500 / 1500	35.3 ± 1.2
	Adam	0.5099 ± 0.0016	0/5	1500 / 1500	34.8 ± 1.2
	L-BFGS	0.5068 ± 0.0015	2/5	1336 / 659	26.2 ± 5.9
Ignite	GD	0.4590 ± 0.0020	0/5	1500 / 706	15.0 ± 1.4
	Momentum	0.4087 ± 0.0024	0/5	1500 / 1500	21.4 ± 4.1
	Adam	0.4191 ± 0.0095	0/5	1500 / 1500	19.3 ± 3.2
	L-BFGS	0.3825 ± 0.0024	0/5	1500 / 728	14.7 ± 1.3
Flink	GD	0.4420 ± 0.0036	0/5	1500 / 184	8.3 ± 0.2
	Momentum	0.3096 ± 0.0044	0/5	1500 / 1500	15.1 ± 4.1
	Adam	0.4712 ± 0.0064	0/5	1500 / 1500	14.8 ± 4.1
	L-BFGS	0.2976 ± 0.0039	0/5	1500 / 725	11.2 ± 1.5

seconds, Eq. (11), or failure. Training executes serially on an Intel i7-12700H under Windows, using one numerical-library thread and Python 3.12.14.

Metrics and checkpoints. Eleven equally spaced objective-call targets use the last accepted vector at or before each target; repeated vectors are allowed, and target zero uses initialization. The checkpoint with the highest validation average precision (AP) is selected, with earliest ties, and its F1 threshold maximizes validation F1. AP uses scikit-learn's `average_precision_score`; ROC-AUC and F1 are supplementary. Constant-score AP equals test prevalence. Final-iterate and selected-checkpoint scores and objectives are both retained.

Means, sample standard deviations and plots show variation across paired initialization seeds. Numerical checks cover gradients, quadratic and Rosenbrock problems, Wolfe conditions and budget accounting, and independent replay checks the saved results.

6 Results and Discussion

Tuning sensitivity. All 192 screening and 48 full-budget finalist runs completed. Full-budget selection changed the short-screen winner in 6 of the 12 project–method combinations (Camel/GD; Ignite/momentum and Adam; Flink/GD, Adam and L-BFGS), and Table 3 reports the resulting settings. The two horizons use checkpoint spacings of 15 and 150 calls. Because the choice changes across horizons, the final comparisons use the longer-stage Apache settings and the screening-derived global settings on OpenStack and Qt.

RQ1: objective reduction and stationarity. L-BFGS has the lowest mean final objective on all three Apache tasks: 0.5068 on Camel, 0.3825 on Ignite and 0.2976 on Flink (Table 4). Two of the 60 main runs attain $R \le 10^{-4}$, both Camel/L-BFGS, and 58 exhaust the objective-call budget. Figure 1 shows objective and residual trajectories across seeds.

The three cost measures rank the methods differently. On Flink, GD spends about 1,316 of its 1,500 objective calls on loss-only backtracking trials, whereas momentum and Adam use every call to compute a gradient, and L-BFGS uses about 775 loss-only and 725 gradient calls. Mean solver times are 8.3 seconds for GD, 11.2 for L-BFGS, 14.8 for Adam and 15.1 for momentum. L-BFGS reaches a mean final loss of 0.2976 versus GD's 0.4420 while taking 2.9 more seconds and computing about four times as many gradients. No single measure therefore orders the methods consistently.

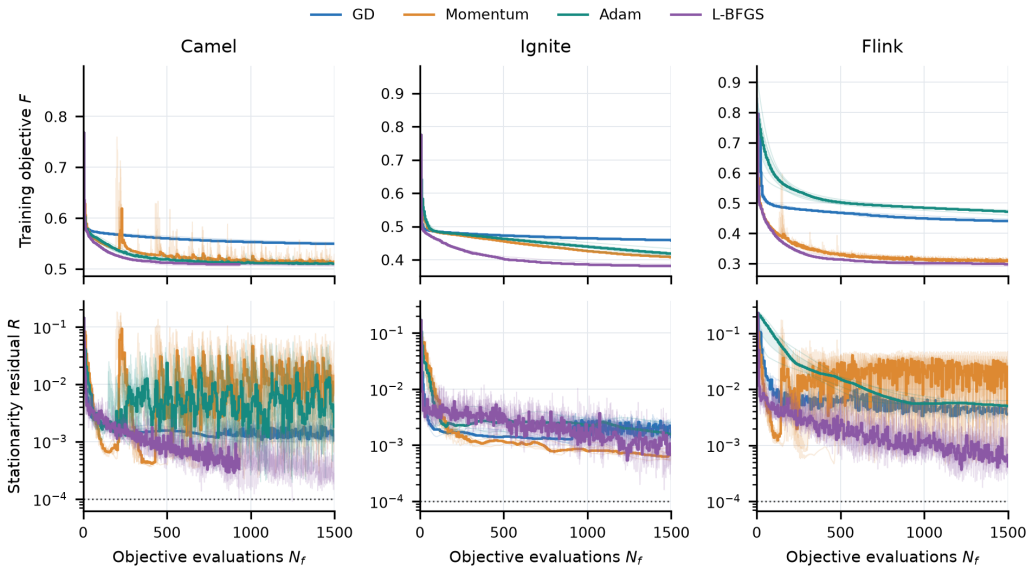


Figure 1: Apache optimization trajectories. Thin curves show initialization seeds; bold curves show medians, and shading shows interquartile ranges. The dotted line marks the gradient stopping tolerance.

Stationarity across thresholds. Across 96 reporting runs, the first-hit counts are 95 at $R \leq 10^{-2}$, 64 at 10^{-3} and five at 10^{-4}; three of the five terminal hits come from width-8 Flink/L-BFGS, and 91 runs use the full evaluation budget. First crossing and final residual differ: among the 21 width-32 momentum runs, 20 cross 10^{-3} and five end below it. Because the momentum direction accumulates velocity, a step can continue through a region where the current gradient norm rises, so first-hit counts measure access to a tolerance while the final residual measures the state delivered at termination.

RQ2: width and initialization. Figure 2 uses exactly the same three seeds at every width. For Adam, increasing width from 8 to 32 raises mean AP from 0.4633 to 0.5304, but increasing it to 128 lowers AP to 0.5014 while reducing the training objective further; the width-32 AP exceeds both alternatives for each of the three paired seeds. L-BFGS reduces mean final objective from 0.4205 through 0.2973 to 0.1412, while mean AP is 0.4638, 0.4617 and 0.4899. All three width-8 runs meet the tolerance, whereas the width-32 and width-128 runs finish above it, and at widths 32 and 128, equal call budgets take approximately 3.8 times longer at width 128 for both optimizers.

The parameter count grows from 113 at width 8 to 449 at width 32 and 1,793 at width 128. Adam peaks at width 32, whereas L-BFGS reaches its highest width-study AP at 128; at width 128, L-BFGS attains a substantially lower final loss than Adam (0.1412 versus 0.4541) while Adam produces the higher checkpoint AP (0.5014 versus 0.4899). The additional parameters therefore strengthen objective fitting and change predictive ranking in different ways for the two update rules.

RQ3: prediction and external replication. Across all five tasks, the optimizer with the lowest mean final objective differs from the optimizer with the highest mean test AP. At validation-selected checkpoints (Figure 3), GD has the largest mean AP on Camel, Ignite, OpenStack and Qt, and Adam leads on Flink; GD and Adam differ by 0.0006 on Ignite (0.7453 versus 0.7447) and by 0.0025 on Qt (0.2165 versus 0.2140). Every task–method mean exceeds its constant-score baseline given by the test prevalence in Table 2, and those baselines span 0.053 on Camel to 0.334 on Ignite. Camel's leading GD AP of 0.203 exceeds its baseline by 0.150, Ignite's leading GD AP of 0.745 exceeds its baseline by 0.411, and GD exceeds the OpenStack and Qt baselines by 0.214 and 0.151.

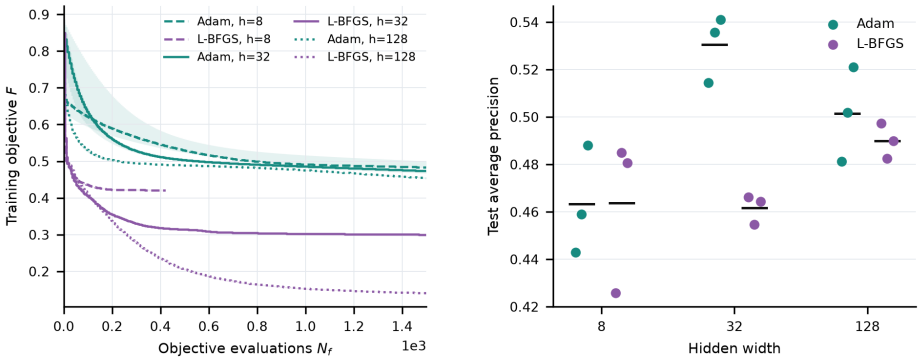


Figure 2: Flink width extension under fixed Adam and L-BFGS settings. The left panel shows median training loss and interquartile range over three paired seeds; the right shows individual test AP and means at validation-selected checkpoints.

Table 5: Test average precision at the validation-selected checkpoint, mean ± sample SD. Apache tasks use five paired initialization seeds; external tasks use three. Bold marks the largest within-task mean using unrounded values.

Task	GD	Momentum	Adam	L-BFGS
Camel	**0.203 ± 0.010**	0.186 ± 0.019	0.196 ± 0.019	0.182 ± 0.003
Ignite	**0.745 ± 0.006**	0.741 ± 0.004	0.745 ± 0.008	0.693 ± 0.029
Flink	0.467 ± 0.015	0.480 ± 0.043	**0.509 ± 0.038**	0.468 ± 0.014
OpenStack	**0.370 ± 0.005**	0.305 ± 0.021	0.353 ± 0.007	0.328 ± 0.010
Qt	**0.217 ± 0.007**	0.196 ± 0.006	0.214 ± 0.010	0.188 ± 0.016

The external tasks reproduce the distinction between optimization and prediction: L-BFGS achieves the lowest mean final objective on OpenStack (0.4085) and Qt (0.5118), but GD has the largest mean AP (0.3705 and 0.2165).

We compare each validation-selected checkpoint with its final iterate from the same run. In 85 of 96 runs, continuing to the final iterate lowers the training objective while lowering test AP; three lower the objective and raise AP, one raises the objective and lowers AP, and seven retain the same point. The paired pattern appears in every task: 16 of 20 Camel runs, 20 of 20 Ignite runs, 27 of 32 Flink runs (including the width extension), 12 of 12 OpenStack runs and 10 of 12 Qt runs lower both loss and AP after the selected checkpoint, with median AP declines of 0.0301, 0.0823, 0.0522, 0.0428 and 0.0377, respectively. L-BFGS alone exhibits this paired decline in 26 of its 27 reporting runs.

For a ranked test set, AP combines precision P_j and recall Q_j at successive score thresholds as $\mathrm{AP} = \sum_j (Q_j - Q_{j-1}) P_j$. Any strictly increasing transformation of every test score preserves the ranking and AP, so the measured AP declines reflect changes in the ordering of positive and negative later commits rather than in the scale of scores. The training objective, by contrast, responds to logit magnitudes and class-weighted errors on earlier commits, so the two targets reward different changes to the predictor. For Apache L-BFGS, the selected checkpoints occur after mean counts of 149, 149 and 180 objective calls on Camel, Ignite and Flink, compared with 1,336, 1,500 and 1,500 calls at termination, so the predictor used for ranking can precede most of the numerical optimization path.

7 Conclusions

We studied whether the training objective is a reliable criterion for two decisions in just-in-time defect prediction, which optimizer to use and when to stop. We formulated a class-weighted one-hidden-layer objective, showed that the zero parameter vector is a strict saddle for all five datasets, and established a first-order stationarity guarantee for Armijo gradient descent. We then compared gradient descent, heavy-ball momentum, Adam and L-BFGS under a matched tuning protocol, with

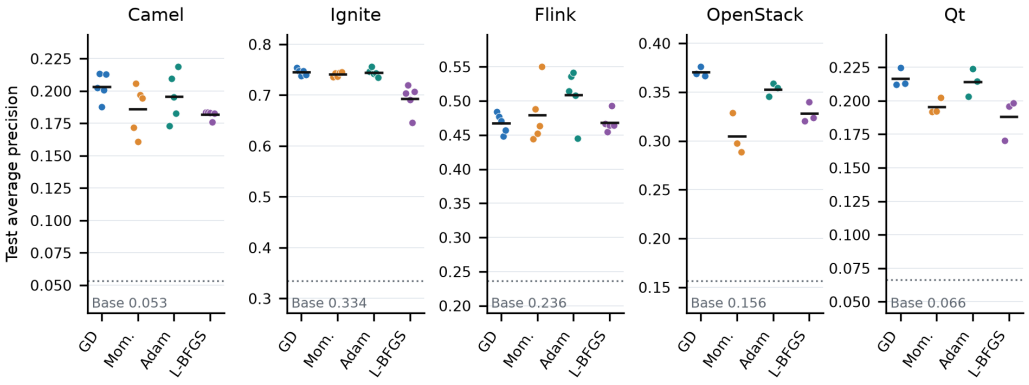


Figure 3: Test AP at validation-selected checkpoints on all five tasks. Dots are paired initialization seeds, black bars show means, and dotted lines show constant-score baselines equal to test prevalence. Axes use separate ranges.

240 tuning and 96 reporting runs on three Apache projects and two external datasets, separating optimization progress, computational cost and defect-ranking performance throughout.

L-BFGS attained the lowest mean final training objective on all five tasks, while test average precision favored gradient descent on four tasks and Adam on Flink. The two criteria therefore produced different optimizer rankings within the same task and tuning protocol, and this disagreement was not attributable to the protocol sensitivity reported in earlier comparisons. The saddle certificate supplied a complementary caution: a vanishing gradient did not certify a minimum, so first-order stationarity should be reported as a separate outcome rather than as evidence of a better fit.

Objective reduction also did not track ranking quality within a single run. In 85 of 96 runs, continuing past the validation-selected checkpoint lowered training loss and test AP alike, and the paired decline appeared in every task, with L-BFGS alone showing it in 26 of its 27 reporting runs. Because average precision is invariant to strictly increasing score transformations, these declines reflect a reordering of positive and negative commits rather than a rescaling of scores, whereas the training objective responds to logit magnitudes and class-weighted errors on earlier commits. The training objective was therefore appropriate for optimizer choice but not for stopping, and validation average precision served as a stopping signal that frequently disagreed with further objective reduction.

Future work. Future work should test whether this separation persists for deeper networks and other ranking metrics, and whether explicit ranking-aware objectives or regularization reduce it. Extending the analysis to stochastic minibatch training and to time-varying project distributions would also clarify when validation AP remains a reliable stopping signal.

Data and Code Availability

All code, the frozen experimental protocol, dependency versions, data hashes, train/validation/test splits, candidate grids, tuning scores, training histories and test predictions are released at [repository URL / footnote]. The repository contains the full pipeline behind the reported results, from preprocessing and label construction through screening, full-budget training, checkpoint selection and evaluation, and `README_STUDY.md` documents the environment, the exact commands for each stage, and the expected outputs, so that the 240 tuning runs and 96 reporting runs can be regenerated.

Saved parameter vectors and per-target evaluation arrays are included, so the reported tables and figures can be reconstructed from stored predictions without retraining. All numerical results were computed from these archived runs.

References

Larry Armijo. Minimization of functions having Lipschitz continuous first partial derivatives. *Pacific Journal of Mathematics*, 16(1):1–3, 1966. doi: 10.2140/pjm.1966.16.1. URL https://msp.org/pjm/1966/16-1/pjm-v16-n1-p01-s.pdf.

Dami Choi, Christopher J. Shallue, Zachary Nado, Jaehoon Lee, Chris J. Maddison, and George E. Dahl. On empirical comparisons of optimizers for deep learning, 2019. URL https://arxiv.org/abs/1910.05446. Preprint; version 3 revised in 2020.

Yuanrui Fan, Xin Xia, Daniel Alencar da Costa, David Lo, Ahmed E. Hassan, and Shanping Li. The impact of mislabeled changes by SZZ on just-in-time defect prediction. *IEEE Transactions on Software Engineering*, 47(8):1559–1586, 2021. doi: 10.1109/TSE.2019.2929761. URL https://yuanrui-fan.github.io/papers/Fan2019-szz.pdf.

Ruinan Jin, Xiao Li, Yaoliang Yu, and Baoxiang Wang. A comprehensive framework for analyzing the convergence of Adam: Bridging the gap with SGD. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 27979–28030. PMLR, 2025. URL https://proceedings.mlr.press/v267/jin25c.html.

Hossein Keshavarz and Meiyappan Nagappan. ApacheJIT: A large dataset for just-in-time defect prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories*, pages 191–195, 2022. doi: 10.1145/3524842.3527996. URL https://arxiv.org/abs/2203.00101.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL https://arxiv.org/abs/1412.6980.

Dong C. Liu and Jorge Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, 45:503–528, 1989. doi: 10.1007/BF01589116. URL https://link.springer.com/article/10.1007/BF01589116.

Shane McIntosh and Yasutaka Kamei. Are fix-inducing changes a moving target? a longitudinal case study of just-in-time defect prediction. *IEEE Transactions on Software Engineering*, 44(5):412–428, 2018. doi: 10.1109/TSE.2017.2693980. URL https://rebels.cs.uwaterloo.ca/papers/tse2017_mcintosh.pdf.

Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964. doi: 10.1016/0041-5553(64)90137-5. URL https://doi.org/10.1016/0041-5553(64)90137-5.

Pratik Rathore, Weimu Lei, Zachary Frangella, Lu Lu, and Madeleine Udell. Challenges in training PINNs: A loss landscape perspective. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 42159–42191. PMLR, 2024. URL https://proceedings.mlr.press/v235/rathore24a.html.

Robin M. Schmidt, Frank Schneider, and Philipp Hennig. Descending through a crowded valley – benchmarking deep learning optimizers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9367–9376. PMLR, 2021. URL https://proceedings.mlr.press/v139/schmidt21a.html.

Bohan Wang, Huishuai Zhang, Qi Meng, Ruoyu Sun, Zhi-Ming Ma, and Wei Chen. On the convergence of Adam under non-uniform smoothness: Separability from SGDM and beyond, 2024. URL https://arxiv.org/abs/2403.15146. Preprint.